



МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ
КАФЕДРА ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

Панькова Ольга Владимировна
Цифровая подпись LESS

КУРСОВАЯ РАБОТА

Научный руководитель:
доцент, к.ф.-м.н.
Чижов Иван Владимирович

Москва, 2026

Оглавление

Введение	3
1 Цели и задачи работы	5
2 Основные математические понятия	5
3 Модель протокола цифровой подписи LESS	7
4 Функция канонической формы CF	9
4.1 Первое сокращение: отказ от матрицы базиса	9
4.2 Второе сокращение: разложение мономиального преобразования	9
4.3 Определение канонической формы	10
4.4 Построение канонической формы	10
5 Ключевые оптимизации	14
5.1 Улучшенный алгоритм канонической формы	14
5.2 Оптимизация сортировки с помощью гистограмм	16
5.3 Повторное использование ведущих элементов в алгоритме приведения матрицы к ступенчатому виду	17
5.4 Оптимизированный RREF	18
5.5 Ослепление	20
5.6 Генерация случайных перестановок за постоянное время	22
6 Результаты	25
7 Приложение	28
7.1 Арифметика в поле $\mathbb{F}_{127}$	28
7.2 Вспомогательные матричные операции	29
7.3 Стандартный RREF	33
7.4 Генерация тестовых матриц	34
7.5 Основные алгоритмы на C	35
Заключение	46

Список литературы

47

Введение

Одной из перспективных пост-квантовых схем цифровой подписи, дошедших до второго раунда конкурса NIST, является схема LESS. Особенность заключается в том, что ее безопасность основана исключительно на проблеме линейной эквивалентности кодов, которая считается сложной даже для квантовых компьютеров.

В первой версии схемы LESS были выявлены два существенных недостатка: большой размер подписи, составляющий от 5 до 8 килобайт, и низкая скорость работы. Для решения проблемы размера подписи было предложено использовать канонические формы, что позволило сократить размер подписи до 1–2.5 килобайт. Однако это улучшение привело к новой проблеме: вычисление канонической формы в примитивной реализации оказалось в 9 раз медленнее операции приведения матрицы к ступенчатому виду. Таким образом, возникла необходимость в оптимизации алгоритмов вычисления канонической формы и других операций.

В данной работе исследуются вопросы, связанные с оптимизацией схемы подписи LESS: рассматриваются предложенные в статье «LESS is Even More» [1] методы ускорения вычислений, включая улучшенный алгоритм канонической формы, замену сортировки сравнением гистограмм, повторное использование ведущих элементов при приведении к ступенчатому виду, а также метод защиты от атак по побочным каналам – ослепление с использованием генерации случайных перестановок за постоянное время. Несколько слов о содержании работы.

В § 1 вводятся цели работы и постановка задачи.

В § 2 даются основные математические определения, необходимые для понимания схемы LESS: поле, в котором работаем, линейные коды, мономиальные преобразования, лексикографический порядок, операции сортировки строк и столбцов матриц.

В § 3 описывается структура 3-раундового протокола с нулевым разглашением, лежащего в основе LESS, и его преобразование в цифровую подпись с помощью трансформации Фиата – Шамира.

В § 4 рассматриваются функция канонической формы и ее упрощения, позволяющие уменьшить размер подписи: отказ от матрицы замены базиса и разложение мономиального преобразования. Также приводится построение

функции канонической формы.

§ 5 посвящён анализу предложенных оптимизаций. В нём рассматриваются улучшенный алгоритм вычисления канонической формы, позволяющий сократить перебор кандидатов за счёт эвристического отбора строк.

В § 6 приводятся результаты экспериментального тестирования производительности базовых и оптимизированных алгоритмов на различных наборах параметров, соответствующих категориям безопасности NIST (LESS-252, LESS-400, LESS-548).

В § 7 приложены необходимые для проверки и тестирования функции.

1. Цели и задачи работы

Целью данной работы является исследование и реализация методов оптимизации цифровой подписи LESS, направленных на ускорение вычисления канонической формы и прочих операций, а также оценка их эффективности по сравнению с базовыми алгоритмами.

Были поставлены и решены следующие задачи:

- 1) Анализ существующих методов сокращения размера подписи.
- 2) Реализация ключевых оптимизаций:
 - повторное использование ведущих элементов при приведении к ступенчатому виду;
 - замена полной сортировки сравнением гистограммами;
 - метод ослепления для защиты от атак по побочным каналам;
 - генерация случайных перестановок за постоянное время с использованием сети Бэтчера.
- 3) Программная реализация на языках Python и C помимо основных алгоритмов: арифметика в поле $\mathbb{F}_{127}$, вспомогательные матричные операции, эталонный алгоритм приведения матрицы к ступенчатому виду.
- 4) Экспериментальное тестирование на наборах параметров LESS-252, LESS-400, LESS-548, соответствующих категориям безопасности NIST.
- 5) Сравнение полученных результатов и достигнутого ускорения.

2. Основные математические понятия

Введем основные математические понятия, используемые в работе.

1. Поля и вектора

Обозначим конечное поле $\mathbb{F}_q$, в работе используется $q = 127$. В общем случае схема LESS допускает использование любого конечного поля $\mathbb{F}_{p^k}$, где p — простое. За $\mathbb{F}_q^*$ будем обозначать конечное поле, в котором отсутствует нулевой вектор. Пусть $\mathbb{F}_q^n$ — векторное пространство размерности n над полем $\mathbb{F}_q$, $\mathbb{F}_q^{k \times n}$ — пространство матриц $k \times n$ над полем $\mathbb{F}_q$. Пусть $\mathbf{v}[i]$ — i -й элемент

вектора, $\mathbf{G}[i, j]$ – элемент матрицы, $\mathbf{G}[i :]$ – i -я строка, $\mathbf{G}[:, j]$ – j -й столбец, $\mathbf{G}[i :] \cdot \mathbf{G}[j :]$ – поэлементное умножение строк.

2. Перестановки и мономиальные отображения

Под S_n будем понимать симметрическую группу на n элементах (перестановки). Для вектора $\mathbf{v} = (v_1, \dots, v_n)$ и перестановки $\pi \in S_n$ введем $\pi(\mathbf{v}) = (v_{\pi^{-1}(1)}, \dots, v_{\pi^{-1}(n)})$. Положим матрицу перестановок $\mathbf{P}$: $\pi(\mathbf{v}) = \mathbf{v} \cdot \mathbf{P}$, мономиальную группу $M_n = \{(\pi, \mathbf{a}) \mid \pi \in S_n, \mathbf{a} \in (\mathbb{F}_q^*)^n\}$, мономиальное отображение: $\mu(\mathbf{v}) = (a_1 \cdot v_{\pi^{-1}(1)}, \dots, a_n \cdot v_{\pi^{-1}(n)})$ и матрицу мономиального отображения $\mathbf{Q}$: $\mu(\mathbf{A}) = \mathbf{G} \cdot \mathbf{Q}$

3. Упорядочивание и сортировка

Символом $\leq_{\mathbb{F}_q}$ будем обозначать полный порядок на поле $\mathbb{F}_q$, тогда функция $\text{Sort} : \mathbb{F}_q^* \rightarrow \mathbb{F}_q^*$ сортирует вектор согласно этому порядку. Для $\mathbb{F}_q^n$ введем лексикографический порядок: $\mathbf{v} \leq_{\mathbb{F}_q^n} \mathbf{v}' \iff \exists i : v_i < v'_i \text{ и } v_j = v'_j \ \forall j < i$. Определим строгий частичный порядок $\prec_{\mathbb{F}_q^n}$ на $\mathbb{F}_q^n$: $\mathbf{v} \prec_{\mathbb{F}_q^n} \mathbf{v}' \iff \mathbf{w} \leq_{\mathbb{F}_q^n} \mathbf{w}'$ и $\mathbf{w} \neq \mathbf{w}'$, где $\mathbf{w} = \text{Sort}(\mathbf{v})$, $\mathbf{w}' = \text{Sort}(\mathbf{v}')$. Также будут необходимы следующие функции:

- $\text{SortRows} : \mathbb{F}_q^{k \times (n-k)} \rightarrow \mathbb{F}_q^{k \times (n-k)}$ — сортировка строк матрицы по $\prec_{\mathbb{F}_q^{n-k}}$
- $\text{SortCols} : \mathbb{F}_q^{k \times (n-k)} \rightarrow \mathbb{F}_q^{k \times (n-k)}$ — сортировка столбцов матрицы по $\leq_{\mathbb{F}_q^k}$

4. Линейные коды и проблема эквивалентности

Пусть $[n, k]$ -код C : k -мерное подпространство $\mathbb{F}_q^n$, тогда порождающая матрица этого кода $\mathbf{G} \in \mathbb{F}_q^{k \times n}$: $C = \{\mathbf{mG} \mid \mathbf{m} \in \mathbb{F}_q^k\}$. Обозначим систематическую форму матрицы: $\mathbf{G} = (\mathbf{I}_k \mid \mathbf{A})$. Введем утверждение, которое является ключевым в задаче цифровой подписи.

Утверждение 2.1. Коды C и C' линейно эквивалентны, если $\exists \mu \in M_n$: $C' = \mu(C)$

Из него исходят следующие проблемы:

- **Проблема эквивалентности кодов:** Найти $\mu \in M_n$ такую, что $C' = \mu(C)$
- **Проблема эквивалентности перестановок** – частный случай проблемы эквивалентности кодов, где μ – перестановка

3. Модель протокола цифровой подписи LESS

Схема цифровой подписи LESS строится на базе 3-раундового протокола с нулевым разглашением, преобразованного в цифровую подпись с помощью трансформации Фиата–Шамира. Это позволяет владельцу секретного ключа доказать знание мономиального отображения между двумя эквивалентными кодами, не раскрывая самого отображения.

Протокол включает в себя три этапа: обязательство, вызов и ответ. Пусть (G, G') — открытый ключ, где $G' = \text{RREF}(\mu(G))$, а $\mu \in M_n$ — секретный ключ.

- 1) **Обязательство.** Доказывающий выбирает случайное мономиальное преобразование $\tilde{\mu} \in M_n$ и вычисляет:

$$(I_k | A) = \text{RREF}(\tilde{\mu}(G)).$$

Затем он вычисляет $\text{cmt} = \text{Hash}(\text{CF}(A))$ и отправляет cmt проверяющему.

- 2) **Вызов.** Проверяющий выбирает случайный бит $\text{ch} \in \{0, 1\}$ и отправляет его доказывающему.
- 3) **Ответ.** В зависимости от значения ch доказывающий формирует ответ:

- Если $\text{ch} = 0$, доказывающий отправляет $\text{rsp} = \tilde{\mu}$.
- Если $\text{ch} = 1$, доказывающий вычисляет композицию $\pi \circ \tilde{\mu} \circ \mu^{-1}$ и извлекает из неё множество J — номера столбцов из G' , которые в $(I_k | A)$ стали единичной матрицей. Ответом служит $\text{rsp} = J$.

Проверяющий верифицирует ответ следующим образом:

- При $\text{ch} = 0$ он вычисляет $\text{RREF}(\text{rsp}(G))$ и проверяет, что хэш от канонической формы полученной матрицы совпадает с cmt .
- При $\text{ch} = 1$ он восстанавливает матрицу A по множеству J :

$$A = (G'_{\text{rsp}})^{-1} \cdot G'_{[n] \setminus \text{rsp}},$$

где G'_{rsp} — столбцы G' с индексами из J , а $G'_{[n] \setminus \text{rsp}}$ — остальные столбцы. Затем проверяется равенство $\text{Hash}(\text{CF}(A)) = \text{cmt}$.

Стоит отметить, что из-за данной композиции $\pi \circ \tilde{\mu} \circ \mu^{-1}$ удастся не раскрывать мономиальное преобразование μ .

Протокол обладает следующими свойствами:

- если доказывающий знает μ , проверяющий всегда принимает доказательство.
- если доказывающий не знает μ , вероятность успешного обмана в каждом раунде не превышает $1/2$. Повторение протокола позволяет экспоненциально снизить эту вероятность. Всегда можно подобрать такое число раундов, чтобы вероятность ошибки была меньше любого порога.
- нулевое разглашение: проверяющий не получает никакой информации о μ помимо факта его существования.

4. Функция канонической формы CF

Напомним постановку задачи. Пусть заданы два линейных кода C и C' с порождающими матрицами $G \in \mathbb{F}_q^{k \times n}$ и $G' \in \mathbb{F}_q^{k \times n}$ соответственно. Требуется доказать эквивалентность этих кодов, то есть установить существование мономиального отображения $\mu \in M_n$ и матрицы замены базиса $S \in \mathbb{F}_q^{k \times k}$ таких, что

$$G' = S \cdot \mu(G).$$

В наивном подходе доказывающий должен был бы предъявить пару (S, μ) , что приводит к большому размеру подписи. Для его сокращения применяются два последовательных упрощения.

4.1. Первое сокращение: отказ от матрицы базиса

Приведение матрицы к ступенчатому виду уничтожает информацию о выборе базиса, поскольку любые две порождающие матрицы одного и того же кода приводятся к одному и тому же ступенчатому виду. Следовательно, вместо проверки равенства $G' = S \cdot \mu(G)$ достаточно проверить условие

$$\text{RREF}(\mu(G)) = \text{RREF}(G').$$

Это позволяет исключить матрицу S из состава подписи, оставив только мономиальное преобразование μ .

4.2. Второе сокращение: разложение мономиального преобразования

Любое мономиальное преобразование μ может быть однозначно представлено в виде композиции двух независимых компонент:

$$\mu = (\pi_{is}, \tau),$$

где:

- π_{is} — перестановка, определяющая, какие k столбцов исходной матрицы станут новым информационным множеством (займут k первых позиций после приведения к систематическому виду);

- $\tau = (\tau_r, \tau_c)$ — мономиальное преобразование, действующее отдельно внутри выбранных k столбцов (τ_r) и оставшихся $n - k$ столбцов (τ_c), то есть выполняющее "внутренние" перестановки и масштабирования этих двух групп.

Если построить функцию канонической формы, инвариантную относительно действия τ , то для доказательства эквивалентности достаточно передать только π_{is} (в виде множества J — номеров столбцов, ставших единичной матрицей). Верификатор, получив π_{is} , вычисляет $CF(\pi_{is}(G))$ и сравнивает с $CF(G')$. Совпадение доказывает эквивалентность кодов.

4.3. Определение канонической формы

Функция канонической формы $CF : \mathbb{F}_q^{k \times (n-k)} \rightarrow \mathbb{F}_q^{k \times (n-k)}$ должна удовлетворять свойству инвариантности, иными словами, каноническая форма не должна зависеть от выбора мономиальных преобразований, применяемых к строкам и столбцам матрицы A :

$$CF(Q_r A Q_c) = CF(A) \quad \forall Q_r \in M_k, \quad \forall Q_c \in M_{n-k},$$

где A — несистематическая часть порождающей матрицы в систематическом виде $(I_k \mid A)$.

4.4. Построение канонической формы

Построение осуществляется в несколько шагов при последовательном добавлении инвариантности к различным типам преобразований.

Шаг 1. Инвариантность к перестановкам

В простейшем случае, когда рассматриваются только перестановки строк и столбцов, каноническая форма строится следующим образом:

$$\text{SortCF}(A) = \text{SortCols}(\text{SortRows}(A)),$$

где SortRows сортирует строки матрицы A в лексикографическом порядке, а затем уже SortCols сортирует столбцы полученной матрицы в лексикографическом порядке, как векторы.

Инвариантность относительно перестановок обеспечивается тем, что перестановка строк или столбцов не меняет их множества, а последующая сортировка выбирает единственный упорядоченный представитель из этого множества.

Шаг 2. Добавление масштабирования строк

Строка v , умноженная на скаляр $\alpha \in \mathbb{F}_q^*$, должна отображаться в тот же канонический вид: для обеспечения инвариантности к умножению строк на ненулевые скаляры вводится операция нормализации строки:

$$\text{ScaleRow}(v) = \left(\sum_{i=1}^{n-k} v_i \right)^{-1} \cdot v,$$

где $v \in \mathbb{F}_q^{n-k}$ — строка матрицы A . Эта операция корректна, если сумма элементов строки не равна нулю.

Для строки $v' = \alpha v$, умноженной на скаляр $\alpha \in \mathbb{F}_q^*$, имеем:

$$\text{ScaleRow}(\alpha v) = \left(\sum_i \alpha v_i \right)^{-1} (\alpha v) = \left(\alpha \sum_i v_i \right)^{-1} (\alpha v) = \left(\sum_i v_i \right)^{-1} v = \text{ScaleRow}(v).$$

Вероятность отказа (ситуация, когда $\sum v_i = 0$) для поля $\mathbb{F}_{127}$ крайне мала и оценивается как $\sim 2^{-80}$, что является приемлемым для практических реализаций.

Шаг 3. Добавление масштабирования столбцов

Чтобы убрать масштабирование со столбца w , можно домножить весь столбец на обратный к любому его ненулевому элементу:

$$\text{ScaleCol}(w, i) = (w_i)^{-1} \cdot w,$$

где w — столбец матрицы, w_i — некоторый ненулевой элемент в этом столбце. Для столбца $w' = \alpha w$:

$$\text{ScaleCol}(\alpha w, i) = (\alpha w_i)^{-1} (\alpha w) = (w_i)^{-1} w = \text{ScaleCol}(w, i).$$

Полный алгоритм построения канонической формы заключается в следующем: для каждого возможного индекса строки $i = 0, 1, \dots, k-1$ матрица

А масштабируется так, чтобы все элементы i -й строки стали равны 1, затем к полученной матрице применяется функция SimpleScaleCF, включающая нормализацию строк и сортировку. Итоговой канонической формой является лексикографически минимальная матрица среди всех k полученных кандидатов.

Алгоритм 1: SortCF

Функция sort_cf реализует каноническую форму, инвариантную относительно перестановок строк и столбцов:

Листинг 1. Алгоритм SortCF

```

#функция сортировки строк в лексикографическом порядке
def sort_rows(matrix: List[List[int]]) -> List[List[int]]:
    return sorted(matrix) #сортировка списков поэлементно

5 #функция сортировки столбцов
def sort_cols(matrix: List[List[int]]) -> List[List[int]]:
    #сортировка столбцов: сначала транспонируем, сортируем как с
    #строки, транспонируем обратно
    transposed = matrix_transposition(matrix)
    sorted_transposed = sort_rows(transposed)
10    return matrix_transposition(sorted_transposed)

def sort_cf(matrix: List[List[int]]) -> List[List[int]]:
    #сначала сортируем строки, потом столбцы
    sorted_by_rows = sort_rows(matrix)
15    result = sort_cols(sorted_by_rows)
    return result

```

Алгоритм 2: SimpleScaleCF

Добавляет инвариантность к масштабированию строк:

Листинг 2. Алгоритм SimpleScaleCF

```

def scale_row(row: List[int]) -> List[int]:
    #анализируем сумму элементов строки - если 0, не трогаем стр
    #оку
    row_sum = 0
    for elem in row:
5        row_sum = gf127_add(row_sum, elem)

```

```

        if row_sum == 0:
            return row
#масштабируем каждый элемент на обратный к сумме
10     scale_factor = gf127_inv(row_sum) #обратный к сумме
        return [gf127_mul(elem, scale_factor) for elem in row]

def simple_scale_cf(matrix: List[List[int]]) -> List[List[
    int]]:
#нормируем строку и применяем алгоритм 1
15     return sort_cf([scale_row(row) for row in matrix])

```

Алгоритм 3: SimpleCF

Полный алгоритм с перебором строк-кандидатов:

Листинг 3. Алгоритм SimpleCF

```

def simple_cf(G: List[List[int]]) -> List[List[int]]:
    k, n = len(G), len(G[0])
    M = [[126] * n for _ in range(k)] #инициализируем матри
цу максимальными элементами поля
    for i in range(k): #перебираем строки для нахождения ведущ
ей
5     #копируем матрицу G
        B = [row[:] for row in G]
        for j in range(k): #перебираем столбцы
#нулевой элемент не трогаем
            if G[i][j] == 0:
10                continue
        #вычисляем обратный элемент к G[i][j] в поле GF(127)
        scale_factor = gf127_inv(G[i][j])
        #масштабируем j-й столбец так, чтобы элемент в строке i стал
1
        for row_idx in range(k):
15            B[row_idx][j] = gf127_mul(B[row_idx][j],
scale_factor)
        #приводим полученную матрицу к канонической форме, инвариант
ной относительно умножения строк на скаляр и перестановок
        B_cf = simple_scale_cf(B)
        #сравниваем полученную каноническую форму с текущей лучшей
        if matrix_less(B_cf, M):
20            M = [row[:] for row in B_cf]
    return M

```

5. Ключевые оптимизации

5.1. Улучшенный алгоритм канонической формы

Базовый алгоритм SimpleCF требует перебора всех k строк матрицы A для выбора индекса i , дающего лексикографически минимальную каноническую форму. Такой полный перебор является основной причиной низкой производительности наивной реализации.

В статье предложен улучшенный алгоритм канонической формы, основанный на следующих наблюдениях: лексикографический порядок матрицы определяется её первой, минимальной строкой, после масштабирования столбцов строка с наибольшим числом нулевых элементов имеет наибольший шанс стать минимальной, поскольку нули в старших разрядах делают вектор "меньше" при лексикографическом сравнении. Основная идея нового предложенного вычисления канонической формы — пропустить как можно больше вычислительных шагов, о которых мы заранее знаем, что они не приведут ни к чему полезному.

Кроме того, строки, уже содержащие нули, часто приводят к нулевой сумме элементов после масштабирования столбцов, что вызывает отказ операции масштабирования строк. Такие строки можно исключить из множества кандидатов на роль масштабирующего индекса i .

Листинг 4. Алгоритм ImprovedCF

```
def improved_cf(G: List[List[int]]) -> List[List[int]]:
    k = len(G)
    n_minus_k = len(G[0])
    #массив числа нулей в каждой из строк
5     zero_counts = [sum(1 for x in row if x == 0) for row in
        G]
    #максимальное число нулей среди строк
    max_zeros = max(zero_counts) if zero_counts else 0
    #множество индексов строк с максимальным числом нулей
    J = [i for i in range(k) if zero_counts[i] == max_zeros]
10    #множество индексов строк, где есть хотя бы 1 ноль
    Z = [i for i in range(k) if zero_counts[i] > 0]
    #инициализируем матрицу максимальными элементами поля
    L = [126] * n_minus_k
    #храним индекс лучшей строки
15    r = 0
```

```

    for i in range(k): #перебираем строки без нулей
        if i in Z:
            continue
#матрица из масштабированных строк с максимальным числом нул
ей
20     B = []
        for j in J: #перебираем строки с максимальным количе
ством нулей
#масштабируем строки с максимальным числом нулей, по сути эт
о масштабирование столбцов, но для каждой строки отдельно
            scaled_row = [gf127_mul(G[j][col], gf127_inv(G[i]
                ][col])) for col in range(n_minus_k)]
            B.append(scaled_row)
25     b, L = sub_scale_cf_preprocess(B, L, len(J))
#если нашли индекс лучшей строки - сохраняем
        if b:
            r = i
    B = []
30 for j in range(k):
#масштабируем все строки с максимальным числом нулей по лу
чшей строке
        if G[r][j] != 0:
            scale_factor = gf127_inv(G[r][j])
            scaled_row = [gf127_mul(G[j][col], scale_factor)
                for col in range(n_minus_k)]
35     else:
            scaled_row = G[j][:]
            B.append(scaled_row)
#если матрица не находится в канонической форме, строим ее
    if not is_scale_cf(B):
40     return simple_cf(G)
    return B

```

Листинг 5. Алгоритм SubScaleCFPreprocess

```

def sub_scale_cf_preprocess(G: List[List[int]], L: List[int]
    ], z: int) -> tuple[bool, List[int]]:
    r = False
    n_minus_k = len(G[0])
    for i in range(z):
5         s = 0
#считаем сумму элементов в строке
        for j in range(n_minus_k):
            s = gf127_add(s, G[i][j])

```

```

10  #нормализация через сумму
    if s != 0:
        s = gf127_inv(s)
        if s == 0:
#иначе используем сумму обратных
            s = 0
15    for j in range(n_minus_k - 1):
        if G[i][j] != 0: #защита от нулей
            s = gf127_add(s, gf127_inv(G[i][j]))
        if s == 0:
            continue
20 #если сумма не 0, можно масштабировать и сравнивать
    T = [gf127_mul(G[i][j], s) for j in range(n_minus_k)]
    if vector_less(T, L):
        L = T[:]
        r = True
25    return r, L

```

5.2. Оптимизация сортировки с помощью гистограмм

Значительная часть вычислительной нагрузки в функции канонической формы приходится на сортировку строк и столбцов матрицы. Реализация сортировки с постоянным временем, необходимая для защиты от атак по побочным каналам, требует существенных затрат.

В статье предложена альтернатива, основанная на использовании гистограмм. Для вектора $v \in \mathbb{F}_q^m$ определим гистограмму $H(v)$ как вектор длины q , i -я компонента которого равна количеству вхождений элемента $i \in \mathbb{F}_q$ в вектор v :

$$H(v)_i = |\{j \mid v_j = i\}|, \quad i \in \mathbb{F}_q.$$

Ключевое наблюдение состоит в том, что лексикографическое сравнение гистограмм $H(v)$ и $H(w)$ эквивалентно лексикографическому сравнению векторов v и w после сортировки. Это позволяет заменить дорогостоящую операцию сортировки сравнением гистограмм, которое может быть реализовано более эффективно.

Для сравнения векторов в лексикографическом порядке используется сортировка подсчётом:

Листинг 6. Алгоритм сортировка подсчётом

```
def sort_vector(v: List[int]) -> List[int]:
```



```

q = 127
C = [0] * q
#подсчёт количества каждого элемента
5 for i in range(len(v)):
    C[v[i]] = C[v[i]] + 1
#создание отсортированного вектора
j = 0
for i in range(q):
10     while C[i] > 0:
        v[j] = i
        C[i] -= 1
        j += 1
    return v

```

Для ускорения сравнения векторов вместо полной сортировки используется гистограмма:

Листинг 7. Алгоритм Histogram

```

def histogram(v: List[int]) -> List[int]:
    q = 127
    w = [0] * q
    for i in range(len(v)):
5         w[v[i]] = w[v[i]] + 1
    return w

```

5.3. Повторное использование ведущих элементов в алгоритме приведения матрицы к ступенчатому виду

Оптимизация приведения матрицы к ступенчатому виду основана на следующем наблюдении. Входная матрица $\mu(G_0)$ получена применением мономиального преобразования μ к матрице G_0 , которая уже находилась в ступенчатом виде. Некоторые столбцы после такого преобразования остаются готовыми ведущими элементами, то есть содержат ровно один ненулевой элемент. Обработку таких столбцов можно пропустить, что сокращает количество операций.

Для параметров LESS, где $k = n/2$, в среднем $k/2$ из k столбцов с ведущими элементами в $\mu(G_0)$ были ведущими элементами и в G_0 . Это позволяет сократить время выполнения алгоритма приблизительно на 50%.

Однако количество наследуемых ведущих элементов зависит от секретной перестановки в μ , что приводит к непостоянному времени выполнения и

создаёт риск утечки информации через побочные каналы. Для решения этой проблемы может вводиться ограничение на число повторно используемых ведущих элементов.

Вероятность того, что количество наследуемых ведущих элементов равно p , определяется следующим распределением:

$$P(p) = \frac{\binom{k}{p} \binom{n-k}{k-p}}{\binom{n}{k}},$$

где k – размерность кода, n – длина кода. Для схемы LESS $k = n/2$, поэтому формула принимает вид:

$$P(p) = \frac{\binom{k}{p} \binom{k}{k-p}}{\binom{2k}{k}} = \frac{\binom{k}{p}^2}{\binom{2k}{k}}.$$

Порог безопасности P выбирается как минимальное целое число, удовлетворяющее условию:

$$P = \min \left\{ p : \sum_{i=0}^{p-1} P(i) < 2^{-64} \right\}.$$

Здесь 2^{-64} – требуемый уровень безопасности согласно рекомендациям NIST. Для параметров схемы LESS, соответствующих трём категориям безопасности, значения порога составляют: $P = 25$, $P = 51$, $P = 79$ соответственно для категорий 1, 3, 5.

Дополнительная защита реализуется путём предварительной перестановки строк, которая размещает унаследованные ведущие элементы на диагонали и минимизирует вероятность их "порчи" в процессе приведения.

5.4. Оптимизированный RREF

Листинг 8. Алгоритм 4: RREF с повторным использованием ведущих элементов

```
def rref_pivot_reuse_algorithm4(G: List[List[int]], p: List[
    bool], l: int) -> List[List[int]]:
    k = len(G)
    n = len(G[0])
    rc = 0 # счётчик переиспользованных ведущих элементов
```

```

5  # шаг 1: перестановка переиспользуемых ведущих элементов
    for col in range(k):
        #если столбец был ведущим элементом изначальной матрицы и ес
        #ли еще можно переиспользовать ведущие элементы
            if col < len(p) and p[col] == 1 and rc < l:
                #первые rc строк уже заняты переставленными ведущими элемент
                #ами
10         for row in range(rc, k):
            if G[row][col] != 0:
                #нашли ведущий элемент - меняем строки местами, чтобы ведущи
                #й элемент был на диагонали
                    G[rc], G[row] = G[row], G[rc]
                    rc += 1
15         break

    #шаг 2: основной алгоритм
    rc = 0 # обнуляем счетчик
    for row_to_reduce in range(k): #строки для приведения
20         i = 0
            j = 0
            found_pivot = False
            #ищем с текущих столбца и строки ведущий элемент
            for j in range(row_to_reduce, n):
25                 for i in range(row_to_reduce, k):
                    if G[i][j] != 0:
                        found_pivot = True
                        break

                if found_pivot:
30                     break
            #ведущий элемент не найден - пропускаем строку
            if not found_pivot:
                continue
            #если ведущий элемент не на диагонали
35             if row_to_reduce != i:
                #перестановка строк портит переиспользуемый ведущий в столбц
                #е i
                    if i < len(p):
                        p[i] = 0 #ведущий элемент больше не переис
                        #пользуется
                #меняем строки местами
40                 G[row_to_reduce], G[i] = G[i], G[row_to_reduce]
            #строка не была с ведущим элементом и счетчик не переполнен
            if row_to_reduce < len(p) and not p[row_to_reduce]

```

```

and rc < 1:
    #нормализация строки
    if G[row_to_reduce][j] != 1:
45         inv_pivot = gf127_inv(G[row_to_reduce][j])
        for col in range(j, n):
            G[row_to_reduce][col] = gf127_mul(G[
row_to_reduce][col], inv_pivot)

    #обнуление остальных строк в столбце
50     for i in range(k):
        if i != row_to_reduce and G[i][j] != 0:
            for col in range(j, n):
                G[i][col] = gf127_sub(G[i][col],
                    gf127_mul(G[i][j], G[
row_to_reduce][col]))
55     else:
#переиспользуем ведущий элемент
        rc += 1
    return G

```

5.5. Ослепление

Для защиты от атак по побочным каналам (например, анализ времени выполнения или потребляемой мощности) при использовании быстрых алгоритмов с переменным временем применяется метод ослепления. Основная цель – обеспечить возможность использования быстрой реализации канонической формы на стороне подписывающего, не раскрывая секретную информацию через побочные каналы.

Суть метода. Перед каждым вычислением канонической формы подписывающий преобразует секретную матрицу A в случайный вид:

$$A_{\text{blind}} = Q_l \cdot A \cdot Q_r,$$

где $Q_l \in M_k$ и $Q_r \in M_{n-k}$ — равномерно случайные мономиальные матрицы, генерируемые заново для каждого раунда. В силу инвариантности канонической формы:

$$\text{CF}(A_{\text{blind}}) = \text{CF}(Q_l \cdot A \cdot Q_r) = \text{CF}(A),$$

следовательно, результат вычислений не изменяется, и подпись остаётся корректной.

Сравнение подходов. Без ослепления атакующий знает $B = CF(A)$ и публичную матрицу A' . Существует связь $A = Q'_r \cdot A' \cdot Q'_c$, где $Q'_r \in M_k$, $Q'_c \in M_{n-k}$ — неизвестные мономиальные матрицы. Если через побочные каналы утечёт информация об A , атакующий сможет восстановить Q'_r и Q'_c , что раскрывает часть секретного преобразования.

При использовании ослепления входом в CF становится A_{blind} . Подставляя выражение для A , получаем:

$$A_{\text{blind}} = Q_l \cdot A \cdot Q_r = Q_l \cdot (Q'_r \cdot A' \cdot Q'_c) \cdot Q_r = (Q_l Q'_r) \cdot A' \cdot (Q'_c Q_r).$$

Поскольку Q_l и Q_r выбраны равномерно случайно, матрицы $Q_l Q'_r$ и $Q'_c Q_r$ также являются равномерно случайными и независимыми от секрета. Следовательно, A_{blind} для атакующего является равномерно случайной матрицей, и любая утечка информации через побочные каналы касается только случайных матриц Q_l и Q_r , не неся информации о секретном преобразовании.

Цена за ослепление составляет два дополнительных мономиальных умножения за раунд. Согласно оценкам авторов, это составляет менее 1% от общего времени выполнения схемы, что делает метод практически бесплатным с точки зрения производительности.

Метод ослепления для защиты от атак по побочным каналам:

Листинг 9. Ослепление

```
def sample_monomial(state: bytes, n: int) -> Tuple[List[int], List[int]]:
    #начальная перестановка - тривиальная
    pi = list(range(n))
    u = [0] * n
5 #u[i] = 1 + случайное_число(0-125) - результат 1-126
    for i in range(n):
        u[i] = 1 + sample_uniform_number(state + f"_coeff_{i}"
                                         ).encode(), 125)
    #генерируем случайную перестановку
    pi = random_permutation(pi, state + b"_permutation")
10 return pi, u

def blind(G: List[List[int]], state: bytes) -> List[List[int]]:
    k = len(G)
    n_minus_k = len(G[0]) if k > 0 else 0
15 #генерируем две случайные мономиальные матрицы
```

```

    Q_r = sample_monomial(state + b"_right", n_minus_k)
    Q_l = sample_monomial(state + b"_left", k)
    #умножение слева и справа
    temp = matrix_multiply_monomial(G, Q_l, True)
20    result = matrix_multiply_monomial(temp, Q_r, False)
    return result

```

5.6. Генерация случайных перестановок за постоянное время

Для безопасной генерации подписи необходимо выбирать случайное мономиальное преобразование μ , содержащее случайную перестановку π порядка n , за постоянное время. Тривиальные алгоритмы генерации случайных перестановок имеют непостоянное время выполнения, что может привести к утечке информации о секретном ключе.

В качестве решения используется сеть перестановок Бэтчера. Сеть состоит из последовательности элементарных компараторов, каждый из которых сравнивает два элемента и, возможно, меняет их местами. Однако вместо сравнения на каждом компараторе "подбрасывается" случайный бит: если бит равен 1, элементы обмениваются местами; если 0 — остаются на своих местах.

Количество случайных битов, необходимых для управления всеми компараторами, составляет:

$$\frac{n}{4} \lfloor \log_2 n \rfloor (\lfloor \log_2 n \rfloor + 1).$$

Такой подход гарантирует, что время выполнения не зависит от генерируемой перестановки, что исключает возможность атак по времени.

Листинг 10. Генерация случайной перестановки за постоянное время

```

def random_permutation(v: List[int], state: bytes) -> List[
    int]:
    n = len(v)
    #работаем с копией, чтобы не изменить исходный массив
    v = v.copy()
5    #счётчик использованных случайных битов
    c = 0
    #количество уровней в сети
    t = math.ceil(math.log2(n)) - 1

```

```

log2n_ceil = math.ceil(math.log2(n))
10 #количество необходимых случайных битов и округление до бай
тов
total_bits = (n // 4) * log2n_ceil * (log2n_ceil + 1)
total_bytes = (total_bits + 7) // 8

#генерация случайных битов через хэш-функцию
15 R_bytes = b''
current_hash = hashlib.sha256(state).digest()
while len(R_bytes) < total_bytes:
    R_bytes += current_hash
    current_hash = hashlib.sha256(current_hash).digest()
20 # надо обрезать до нужного числа байтов
R_bytes = R_bytes[:total_bytes]

#преобразование байтов в биты
R_bits = []
25 for byte in R_bytes:
    for bit_pos in range(8):
        R_bits.append((byte >> bit_pos) & 1)
#обрезаем до нужного количества битов
R_bits = R_bits[:total_bits]

30 #степени двойки от 2^t до 1
p_values = [1 << (t - i) for i in range(t + 1)]
for p in p_values:
    r = 0
35 #начальное расстояние между сравниваемыми элементами
d = p
#степени двойки от 2^t до p
q_values = [1 << (t - i) for i in range(t + 1) if (1
<< (t - i)) >= p]
for q in q_values:
40 #сравниваем и меняем местами элементы на расстоянии d
for i in range(n - d):
#определяем, какие пары элементов сравнивать, чтобы каждая п
ара сравнивалась ровно один раз
if (i & p) == r and c < len(R_bits) and
R_bits[c] == 1:
    v[i], v[i + d] = v[i + d], v[i]
45 c += 1
r = p
d = q - p

```

```
|     return v
```


6. Результаты

Авторы статьи используют AMD Ryzen 5 7600X с частотой 4.7 ГГц, у меня же процессор AMD Ryzen 5 5500U с частотой 2.1ГГц. Так как язык Python значительно медленнее C, алгоритмы реализованы на обоих языках, Python использован для более простой читаемости основных алгоритмов и общего сравнения ускорений различных алгоритмов, C – для сравнения работы матриц алгоритмов для больших матриц.

Таблица 1. Сравнение CF original и CF improved (тыс. циклов)

Алгоритм	LESS-252	LESS-400	LESS-548
CF original	2472	9010	21130
CF improved	294	676	768
Ускорение	8.4x	13.3x	27.5x

Таблица 2. Сравнение SORT и HISTOGRAM (тыс. циклов)

Алгоритм	LESS-252	LESS-400	LESS-548
SORT	0.85	0.97	1.16
HISTOGRAM	0.23	0.31	0.39
Ускорение	3.77x	3.18x	2.98x

Таблица 3. Сравнение RREF оригинальный и оптимизированный (тыс. циклов)

Алгоритм	LESS-252	LESS-400	LESS-548
RREF original	2677.80	10593.32	26936.07
RREF opt	709.63	2750.04	7048.22
Ускорение	3.77x	3.85x	3.82x

Достигнутые значения ускорения сопоставимы с данными оригинальной статьи, а также все реализованные оптимизации являются эффективными и подтверждают результаты, полученные авторами.

Экспериментальная проверка применения реализованных алгоритмов в генерации ключа, подписывании и верификации проводилась на основе кода из открытого репозитория схемы LESS. В данный код были интегрированы все разработанные в рамках работы оптимизированные функции. Такой подход позволил провести корректное сравнение производительности базовой и оптимизированной реализаций в идентичных условиях.

Таблица 4. Сводные результаты NIST Category 1, 3, 5 (в миллионах циклов)

Набор	Операция	Статья неопт.	Статья опт.	Моя неопт.	Моя опт.
LESS-252-192	KeyGen	0.9	0.7	3.2	3.0
	Sign	1001.8	117.0	3871.9	1028.0
	Verify	970.7	106.8	2797.9	606.1
LESS-252-68	KeyGen	1.7	1.6	8.7	7.0
	Sign	353.5	41.4	1133.6	349.7
	Verify	351.9	39.3	1029.7	215.8
LESS-252-45	KeyGen	3.6	3.6	15.6	15.6
	Sign	233.5	26.5	743.1	232.4
	Verify	233.3	26.3	663.5	146.2
LESS-400-220	KeyGen	1.1	1.0	8.0	7.0
	Sign	2135.6	325.7	6776.1	2638.3
	Verify	2162.7	317.1	6009.5	1780.7
LESS-400-102	KeyGen	4.8	4.8	22.7	22.0
	Sign	1001.7	162.5	3121.7	1182.7
	Verify	970.7	152.9	2776.7	822.4
LESS-548-345	KeyGen	4.2	4.1	17.1	17.4
	Sign	5026.9	1234.9	16970.9	8511.3
	Verify	4960.3	1172.0	14727.8	6096.1
LESS-548-137	KeyGen	10.7	10.6	47.1	47.1
	Sign	1981.3	485.8	6831.1	3389.1
	Verify	1973.3	481.1	5990.0	2421.0

Вывод

Полученные результаты показывают, что предложенные оптимизации позволяют значительно ускорить операции подписи и верификации. Для набора LESS-252-192 ускорение подписания составило 3.8 раза, верификации – 4.6 раза. Для набора LESS-252-68 ускорение подписания достигло 3.2 раза, верификации – 4.8 раза. Для набора LESS-252-45 ускорение подписания – 3.2 раза, верификации – 4.5 раза. Для наборов категории 3 ускорение подписи составило 2.6 раза, верификации – 3.4 раза. Для LESS-548-345 ускорение подписи составило 2 раза, верификации – 2.4 раза. Для LESS-548-137 ускорение подписи – 2 раза, верификации – 2.5 раза.

Операция генерации ключей практически не изменилась после внедрения оптимизаций, что объясняется тем, что keygen не использует каноническую форму. Небольшие различия объясняются погрешностью измерений.

Сравнение с данными оригинальной статьи демонстрирует, что абсолютное время выполнения в моей работе выше. Предполагаю, что это объясняется следующими факторами: различием в аппаратном обеспечении и отсутствием AVX2 арифметики в данной реализации. Тем не менее, относительное ускорение, достигаемое за счёт предложенных оптимизаций, является существенным и подтверждает эффективность разработанных методов. А

также порядок числа циклов соответствует порядку из статьи, что подтверждает корректность реализованных оптимизаций.

7. Приложение

Ниже приведены реализации основных алгоритмов на языке Python и C.

7.1. Арифметика в поле $\mathbb{F}_{127}$

В схеме LESS используется конечное поле $\mathbb{F}_{127}$ с элементами $\{0, 1, \dots, 126\}$. Операции сложения, вычитания и умножения выполняются по модулю 127:

Листинг 11. Определение макросов и импортирование библиотек

```
#include <stdint.h>    // uint8_t, uint16_t, uint64_t
#include <stdbool.h>    // bool, true, false
#include <stdlib.h>     // malloc, calloc, free
#include <string.h>     // memcpy, memset
5 #include <stdio.h>    // printf, fprintf, fflush
#define GF127_MOD 127
```

Листинг 12. Арифметические операции в поле $\mathbb{F}_{127}$

```
def gf127_add(a: int, b: int) -> int:
    s = a + b
    return s - 127 if s >= 127 else s

5 def gf127_sub(a: int, b: int) -> int:
    d = a - b
    return d + 127 if d < 0 else d

10 def gf127_mul(a: int, b: int) -> int:
    x = a * b
    # (x >> 7) + (x & 127) заменяет взятие по модулю 127
    t = (x >> 7) + (x & 127)
15 return t - 127 if t >= 127 else t

#по малой теореме Ферма: a^{p-2} = a^{-1} (mod p)
def gf127_inv(a: int) -> int:
20 return pow(a, 125, 127) # 127 - 2 = 125
```

Листинг 13. Арифметические операции в поле $\mathbb{F}_{127}$

```

gf127_t gf127_add(gf127_t a, gf127_t b) {
    uint8_t s = a + b;
    return (s >= GF127_MOD) ? s - GF127_MOD : s;
}
5
gf127_t gf127_sub(gf127_t a, gf127_t b) {
    int16_t d = a - b;
    return (d < 0) ? d + GF127_MOD : d;
}
10
gf127_t gf127_mul(gf127_t a, gf127_t b) {
    uint16_t x = a * b;
    uint8_t t = (x >> 7) + (x & 127);
    return (t >= GF127_MOD) ? t - GF127_MOD : t;
15 }

gf127_t gf127_inv(gf127_t a) {
    gf127_t res = 1;
    int exp = 125;
20 while (exp > 0) {
        if (exp & 1) res = gf127_mul(res, a);
        a = gf127_mul(a, a);
        exp >>= 1;
    }
25 return res;
}

```

7.2. Вспомогательные матричные операции

Листинг 14. Вспомогательные функции для работы с матрицами и векторами

```

def matrix_create(rows: int, cols: int, val=0) -> List[List[
    int]]:
    #создание матрицы размера rows * cols.
    return [[val for j in range(cols)] for i in range(rows)]
5

def matrix_transposition(matrix: List[List[int]]) -> List[
    List[int]]:

```

```

        return [[matrix[j][i] for j in range(len(matrix))] for i
in range(len(matrix[0]))]

10 def matrix_less(A: List[List[int]], B: List[List[int]]) ->
    bool:
    #лексикографическое сравнение матриц,
    #сравнение происходит построчно, внутри строки - поэлементно
        for i in range(len(A)):
            for j in range(len(A[0])):
15                 if A[i][j] < B[i][j]:
                        return True
                    elif A[i][j] > B[i][j]:
                        return False
            return False

20 def sort_vector(v: List[int]) -> List[int]:
    q = 127
    #подсчёт количества каждого элемента
    counts = [0] * q
25     for x in v:
        counts[x] += 1

    #восстановление отсортированного вектора
    result = []
30     for i in range(q):
        result.extend([i] * counts[i])
    return result

def vector_less(v1: List[int], v2: List[int]) -> bool:
35 #сортируем оба вектора
    sorted_v1 = sort_vector(v1)
    sorted_v2 = sort_vector(v2)

    #поэлементное сравнение отсортированных векторов
40     for i in range(len(sorted_v1)):
        if sorted_v1[i] < sorted_v2[i]:
            return True
        elif sorted_v1[i] > sorted_v2[i]:
            return False
45     return False

```

Листинг 15. Вспомогательные алгоритмы на С

```

//создание матрицы с выделением памяти
matrix_t* matrix_create(int rows, int cols) {
    matrix_t* m = (matrix_t*)malloc(sizeof(matrix_t)); //выд
    еляем память под структуру
    m->data = (gf127_t*)calloc(rows * cols, sizeof(gf127_t))
    ; //обнуляем данные
5    m->rows = rows;
    m->cols = cols;
    return m;
}

10 //освобождение памяти матрицы
void matrix_free(matrix_t* m) {
    if (m) {
        free(m->data);
        free(m);
15    }
}

//копирование матрицы
matrix_t* matrix_copy(matrix_t* src) {
20    matrix_t* dst = matrix_create(src->rows, src->cols);
    memcpy(dst->data, src->data, src->rows * src->cols *
    sizeof(gf127_t));
    return dst;
}

25 //транспонирование матрицы
matrix_t* matrix_transposition(matrix_t* m) {
    matrix_t* res = matrix_create(m->cols, m->rows);
    for (int i = 0; i < m->rows; i++) {
        for (int j = 0; j < m->cols; j++) {
30    #j * rows + i - транспонированное положение
        res->data[j * m->rows + i] = m->data[i * m->cols
+ j];
        }
    }
    return res;
35 }

//лексикографическое сравнение матриц

```

```

bool matrix_less(matrix_t* A, matrix_t* B) {
    for (int i = 0; i < A->rows; i++) {
40         for (int j = 0; j < A->cols; j++) {
            gf127_t a = A->data[i * A->cols + j];
            gf127_t b = B->data[i * B->cols + j];
            if (a < b) return true;
            if (a > b) return false;
45         }
    }
    return false; //матрицы равны
}

50 //сортировка вектора подсчётом
void sort_vector(gf127_t* v, int n) {
    int C[GF127] = {0};
    for (int i = 0; i < n; i++) C[v[i]]++;
    int j = 0;
55     for (int i = 0; i < GF127; i++) {
        while (C[i] > 0) {
            v[j++] = i;
            C[i]--;
        }
60     }
}

//сравнение векторов после сортировки
bool vector_less(gf127_t* v1, gf127_t* v2, int n) {
65 //создаём копии
    gf127_t* s1 = (gf127_t*)malloc(n * sizeof(gf127_t));
    gf127_t* s2 = (gf127_t*)malloc(n * sizeof(gf127_t));
    memcpy(s1, v1, n * sizeof(gf127_t));
    memcpy(s2, v2, n * sizeof(gf127_t));
70
    //сортируем копии
    sort_vector(s1, n);
    sort_vector(s2, n);

75 //поэлементное сравнение
    for (int i = 0; i < n; i++) {
        if (s1[i] < s2[i]) { free(s1); free(s2); return true
; }
        if (s1[i] > s2[i]) { free(s1); free(s2); return

```



```

        false; }
    }
80    free(s1); free(s2);
        return false;
}

```

7.3. Стандартный RREF

Для оценки эффективности оптимизированного алгоритма необходимо иметь эталонную реализацию. Функция `rref_standard` реализует классический алгоритм приведения матрицы к ступенчатому виду:

Листинг 16. Стандартный алгоритм RREF

```

def rref_standard(G: List[List[int]]) -> List[List[int]]:
    #Алгоритм:
    #      1. Для каждой строки ищем ведущий элемент
    #      2. Нормализуем строку (делаем ведущий элемент = 1)
    5 #      3. Обнуляем все остальные строки в столбце ведущего
        элемента
    #      4. Переходим к следующей строке и столбцу
        k = len(G)
        n = len(G[0])

    10    lead = 0 #текущий ведущий столбец
        for r in range(k):
            if lead >= n:
                break

    15 #поиск строки с ненулевым элементом в столбце lead
            i = r
            while i < k and G[i][lead] == 0:
                i += 1

    20 #если ведущий элемент не найден, переходим к следующему стол
        бцу
            if i == k:
                lead += 1
                continue

    25 #перестановка строк: найденную строку ставим на позицию r
        G[r], G[i] = G[i], G[r]

```

```

#нормализация строки
    pivot = G[r][lead]
30    if pivot != 1:
        inv_pivot = gf127_inv(pivot)
        for col in range(lead, n):
            G[r][col] = gf127_mul(G[r][col], inv_pivot)

35 #обнуление всех остальных строк в столбце lead
    for i in range(k):
        if i != r and G[i][lead] != 0:
            factor = G[i][lead]
            for col in range(lead, n):
40                G[i][col] = gf127_sub(G[i][col],
                                           gf127_mul(factor,
G[r][col]))

        lead += 1

45    return G

```

7.4. Генерация тестовых матриц

Для экспериментальной оценки производительности необходимо генерировать тестовые матрицы, аналогичные используемым в схеме LESS:

Листинг 17. Импорты для генерации тестовых матриц

```

import random
from typing import List, Tuple

```

Листинг 18. Генерация тестовых матриц в стиле LESS

```

def create_test_matrix_less_style(k: int, n_minus_k: int) ->
    Tuple[List[List[int]], List[bool]]:

    n = k + n_minus_k
    #G = (I_k | A)
5 #создание матрицы A (правая часть)
    A = [[0] * n_minus_k for _ in range(k)]
    for i in range(k):
        for j in range(n_minus_k):

```

```

        if random.random() < 0.1:  #плотность матрицы -
10      10% - стандарт LESS
            A[i][j] = random.randint(1, 126)

#создание матрицы (I_k | A)
    G = [[0] * n for _ in range(k)]
    for i in range(k):
15      G[i][i] = 1  #левая часть - единичная матрица
        for j in range(n_minus_k):
            G[i][k + j] = A[i][j]

#применение случайной перестановки столбцов
20      permutation = list(range(n))
        random.shuffle(permutation)

    G_permuted = [[0] * n for _ in range(k)]
    for i in range(k):
25      for new_j, old_j in enumerate(permutation):
            G_permuted[i][new_j] = G[i][old_j]

#определение, какие столбцы были ведущими элементами в исход
ной матрице
    inverse_perm = [0] * n
30      for new_pos, old_pos in enumerate(permutation):
            inverse_perm[old_pos] = new_pos

    p = [0] * n
    for i in range(k):  #первые k столбцов в исходной были в
        ведущими элементами
35      p[inverse_perm[i]] = 1

    return G_permuted, p

```

7.5. Основные алгоритмы на C

Алгоритм 1: SortCF

Листинг 19. Алгоритм SortCF

```

matrix_t* sort_cf(matrix_t* G) {
    matrix_t* res = matrix_copy(G);

```

```

    if (!res) return NULL;
    sort_rows(res);
5   sort_cols(res);
    return res;
}

```

Алгоритм 2: SimpleScaleCF

//масштабирует строку так, чтобы сумма её элементов стала равна 1.

Листинг 20. Алгоритм SimpleScaleCF

```

void scale_row(gf127_t* row, int n) {
    gf127_t sum = 0;
    //считаем сумму элементов строки
    for (int j = 0; j < n; j++) {
5       sum = gf127_add(sum, row[j]);
    }
    if (sum == 0) return;
    gf127_t inv = gf127_inv(sum);
    for (int j = 0; j < n; j++) {
10   //умножаем каждый элемент строки на обратный, если он существует
        row[j] = gf127_mul(row[j], inv);
    }
}

15 matrix_t* simple_scale_cf(matrix_t* G) {
    matrix_t* res = matrix_copy(G);
    if (!res) return NULL;

    for (int i = 0; i < res->rows; i++) {
20   //масштабируем строки
        scale_row(&res->data[i * res->cols], res->cols);
    }
    matrix_t* sorted = sort_cf(res);
    matrix_free(res);
25   return sorted;
}

```

Алгоритм 3: SimpleCF

Листинг 21. Алгоритм SimpleCF

```

matrix_t* simple_cf(matrix_t* G) {
    int k = G->rows;
    int n = G->cols;

5    matrix_t* M = matrix_create(k, n);
    if (!M) return NULL;

    // Инициализируем максимальными значениями
    for (int i = 0; i < k * n; i++) {
10        M->data[i] = GF127_MOD - 1;
    }

    for (int i = 0; i < k; i++) {
        for (int j = 0; j < k; j++) {
15            if (G->data[i * n + j] == 0) continue;

            //копируем и масштабируем столбец на обратный элемент
            gf127_t scale = gf127_inv(G->data[i * n + j]);
            matrix_t* B = matrix_copy(G);
20            if (!B) continue;

            for (int r = 0; r < k; r++) {
                B->data[r * n + j] = gf127_mul(B->data[r * n
+ j], scale);
            }

25            matrix_t* Bcf = simple_scale_cf(B);
            //если меньше текущего лучшего
            if (Bcf && matrix_less(Bcf, M)) {
                matrix_free(M);
30                M = matrix_copy(Bcf);
            }
            if (Bcf){
                matrix_free(Bcf);
                matrix_free(B);
35        }
        }
    }
    return M;
}

```

}

Алгоритм 4: RREF с повторным использованием ведущих элементов

Листинг 22. RREF с повторным использованием ведущих элементов

```

matrix_t* rref_pivot_reuse(matrix_t* G, bool* p, int l) {
    matrix_t* res = matrix_copy(G);
    if (!res) return NULL;

5   int k = res->rows;
    int n = res->cols;
    int rc = 0;

    // Шаг 1: предварительная перестановка повторно используе
    // мых ведущих элементов
10   for (int col = 0; col < k && rc < l; col++) {
        if (p[col]) {
            for (int row = rc; row < k; row++) {
                if (res->data[row * n + col] != 0) {
                    if (row != rc) {
15                     for (int j = 0; j < n; j++) {
                        gf127_t tmp = res->data[rc * n +
j];
                        res->data[rc * n + j] = res->
data[row * n + j];
                        res->data[row * n + j] = tmp;
                                }
                                }
                                rc++;
                                break;
                                }
                                }
20   }
    }

    // Шаг 2: основной алгоритм
    rc = 0;
30   for (int row_to_reduce = 0; row_to_reduce < k;
row_to_reduce++) {

```

```

        // Поиск ведущего элемента
        int i = 0;
        int j = 0;
//поиск ведущего элемента
35         int found = 0;
        for (j = row_to_reduce; j < n && !found; j++) {
            for (i = row_to_reduce; i < k && !found; i++) {
                if (res->data[i * n + j] > 0) {
                    found = 1;
40                }
            }
        }
        if (!found) continue;

        i--;
45        j--;

        //перестановка строк
        if (row_to_reduce != i) {
            p[i] = 0;
50        }

//меняем местами
        for (int j = 0; j < n; j++) {
            gf127_t tmp = res->data[i * n + j];
            res->data[i * n + j] = res->data[row_to_reduce *
n + j];
55            res->data[row_to_reduce * n + j] = tmp;
        }

        // можно ли повторно использовать ведущий элемент?
        if (!p[row_to_reduce] && rc < 1) {
60 //нормализация строки
            gf127_t pivot = res->data[row_to_reduce * n + j];
            if (pivot != 1) {
                gf127_t inv = gf127_inv(pivot);
                for (int t = j; t < n; t++) {
50                    res->data[row_to_reduce * n + t] = gf127_mul(
res->data[row_to_reduce * n + t], inv);
65                }
            }
        }

//обнуление остальных строк в столбце
        for (int r = 0; r < k; r++) {

```

```

70         if (r != row_to_reduce && res->data[r * n + j] !=
0) {
            gf127_t factor = res->data[r * n + j];
            for (int t = j; t < n; t++) {
                res->data[r * n + t] = gf127_sub(res->data
[r * n + t],
                                                    gf127_mul
(
factor, res->data[row_to_reduce * n + t]));
75                }
            }
        }
//иначе переиспользуем
    } else {
80        rc++;
    }
    }
    return res;
}

```

Алгоритм 5: ImprovedCF

Листинг 23. Улучшенный алгоритм канонической формы ImprovedCF

```

//функция подсчета нулей в каждой строке
int* count_zeros(const matrix_t* G) {
    int* counts = (int*)calloc(G->rows, sizeof(int));
    if (!counts) return NULL;
5
    for (int i = 0; i < G->rows; i++) {
        for (int j = 0; j < G->cols; j++) {
            if (G->data[i * G->cols + j] == 0) {
                counts[i]++;
10            }
        }
    }
    return counts;
}
15
matrix_t* improved_cf(matrix_t* G) {
    int k = G->rows;
    int n = G->cols;

```



```

20 //число нулей в каждой строке
    int* zeros = count_zeros(G);
    if (!zeros) return NULL;

25 //максимальное число нулей по всем строкам
    int max_zeros = 0;
    for (int i = 0; i < k; i++) {
        if (zeros[i] > max_zeros) {
            max_zeros = zeros[i];
30     }
    }

    // J - индексы строк с максимальным числом нулей
    int* J = (int*)malloc(k * sizeof(int));
35     if (!J) {
        free(zeros);
        return NULL;
    }

    int cnt_zeros = 0; //сколько таких строк
40     for (int i = 0; i < k; i++) {
        if (zeros[i] == max_zeros) J[cnt_zeros++] = i;
    }

    // Z - массив: есть ли хоть один ноль в строке
45     bool* Z = (bool*)calloc(k, sizeof(bool));
    if (!Z) {
        free(zeros); free(J);
        return NULL;
    }

50     for (int i = 0; i < k; i++) {
        Z[i] = (zeros[i] > 0);
    }

    // массив с максимальными элементами
    gf127_t* L = (gf127_t*)malloc(n * sizeof(gf127_t));
55     if (!L) {
        free(zeros); free(J); free(Z);
        return NULL;
    }

    for (int j = 0; j < n; j++) L[j] = GF127_MOD - 1;
60

```

```

    int r = 0; //строка, давшая лучший результат
    int found = 0;

    //перебор строк-кандидатов без нулей
65     for (int i = 0; i < k; i++) {
        if (Z[i]) { //пропускаем строки с нулями
            continue;
        }

70     //обращаем строку
    gf127_t* row_inv = (gf127_t*)malloc(n * sizeof(gf127_t));
        if (!row_inv) {
            free(row_inv);
            continue;
75     }

        for (int j = 0; j < n; j++) {
            if (G->data[i * n + j] != 0) {
                row_inv[j] = gf127_inv(G->data[i * n + j]);
            } else {
80                 row_inv[j] = 0;
            }
        }

        // масштабируем строки кандидаты
85     matrix_t* B = matrix_create(cnt_zeros, n);
        if (!B) {
            continue;
        }

90     for (int jj = 0; jj < cnt_zeros; jj++) {
        int row = J[jj];
        for (int c = 0; c < n; c++) {
            B->data[jj * n + c] = gf127_mul(G->data[row
* n + c], row_inv[c]);
        }
95     }

    //нормализуем строки и ищем лучшего кандидата
        if (sub_scale_cf_preprocess(B, L, cnt_zeros)) {
            r = i;
100            found = 1;
        }
    }

```

```

        matrix_free(B);
    free(row_inv);
}
105     matrix_t* B = matrix_create(k, n);
    //если нашли хорошего кандидата - вычисляем полную каноничес
    кую форму
    if (found) {
        gf127_t* row_inv = (gf127_t*)malloc(n * sizeof(gf127_t));
        if (!row_inv) {
110             free(row_inv);
            continue;
        }
        for (int j = 0; j < n; j++) {
            if (G->data[r * n + j] != 0) {
115                 row_inv[j] = gf127_inv(G->data[r * n + j]);
            } else {
                row_inv[j] = 0;
            }
        }
        for (int j = 0; j < k; j++) {
120             for (int c = 0; c < n; c++) {
                B->data[j * n + c] = gf127_mul(G->data[j * n
                + c], row_inv[c]);
            }
        }
125     matrix_t* scaled = simple_scale_cf(B);
        free(zeros);
        free(J);
        free(Z);
        free(L);
130     free(row_inv);
        if (!scaled) {
            return simple_cf(G);
        }
    }
135     return B;
}

```

Алгоритм 6: SubScaleCFPreprocess

Листинг 24. Вспомогательный алгоритм SubScaleCFPreprocess

```

int sub_scale_cf_preprocess(matrix_t* G, gf127_t* L, int z)
{
    int found = 0;
    int n = G->cols;
    gf127_t* T = (gf127_t*)malloc(n * sizeof(gf127_t));
5    if (!T) return 0;

    for (int i = 0; i < z; i++) {
        gf127_t s = 0;
        for (int j = 0; j < n; j++) {
10            s = gf127_add(s, G->data[i * n + j]);
        }
        if (s != 0) {
            s = gf127_inv(s);
        }
15        else {
            s = 0;
            for (int j = 0; j < n - 1; j++) {
                if (G->data[i * n + j] != 0) {
                    s = gf127_add(s, gf127_inv(G->data[i * n
20 + j]]));
                }
            }
            if (s == 0) continue;
        }

        for (int j = 0; j < n; j++) {
25            T[j] = gf127_mul(s, G->data[i * n + j]);
        }

        //сравниваем L и T
        int less = 0;
        for (int j = 0; j < n; j++) {
30            if (T[j] < L[j]) {
                less = 1;
                break;
            }
            if (T[j] > L[j]) break;
35        }

        if (less) {
            for (int j = 0; j < n; j++) L[j] = T[j];
            found = 1;
        }
    }
}

```

```
40         }  
    }  
  
    free(T);  
    return found;  
45 }
```

Заключение

В ходе выполнения данной курсовой работы были достигнуты следующие результаты:

- 1) Проведён анализ схемы цифровой подписи LESS: рассмотрены её математическая основа, структура протокола. В ходе реализации алгоритмов были выявлены и исправлены ошибки в исходных алгоритмах (псевдокодах), приведённых в статье.
- 2) Реализованы ключевые оптимизации, предложенные в статье «LESS is Even More» [1]: улучшенный алгоритм канонической формы, повторное использование ведущих элементов при приведении матрицы к ступенчатому виду, замена сортировки сравнением гистограммами, метод ослепления для защиты от атак по побочным каналам с использованием алгоритма генерации случайных перестановок за постоянное время на основе сети Бэтчера.
- 3) Проведено экспериментальное тестирование производительности базовых и оптимизированных алгоритмов на наборах параметров LESS-252, LESS-400 и LESS-548, соответствующих категориям безопасности NIST. Удалось получить ускорение алгоритмов подписания и верификации.

В качестве продолжения исследования, уже в дипломной работе, планируется реализовать атаку на схему цифровой подписи LESS. Выбор конкретного типа атаки будет произведён после детального изучения существующих методов криптоанализа схемы и анализа её потенциальных уязвимостей.

Список литературы

1. *Beckwith L.* LESS is Even More: Optimizing Digital Signatures from Code Equivalence // JPL DSN Progress Report. — 2025. — Т. 44. — С. 1—23.